Enterprise Student Record & Academic Management Platform

DevOps Foundation, Version Control, Continuous Integration & Container Orchestration
Document 1: Theoretical and Architectural Reference

Document Attribute	Specification Details
Student	Gaurav Borle | PRN: 2125UDSM1071
Course & Unit	UNIT 3: DevOps, CI/CD, Containerization & Application Deployment
Student Project	Enterprise Student Record & Academic Management Platform (Python / Flask / Pytest / Jenkins / Docker / Minikube)
Trainer Reference System	Enterprise Smart Library Management Platform
Source Repository	https://github.com/GauravBorle04/Devops-Miniprojects-SY
Infrastructure	AWS EC2 (Ubuntu), accessed through MobaXterm over SSH
Document Purpose	Theory and design reference explaining the concepts behind each tool used in the project

1. Introduction & Engineering Context

Writing a program that works on one laptop is only a small part of software engineering. The harder part is getting that program built, tested, packaged and running reliably for real users, again and again, without breaking anything. DevOps is the discipline that closes this gap. It joins development (Dev) and IT operations (Ops) into one workflow in which almost every repeatable step is automated and described as code.

This document explains the theory behind the four pillars of that workflow and shows how each one is applied in our project: **Git and GitHub** for version control, **Jenkins** for continuous integration, **Docker** for packaging, and **Kubernetes (Minikube)** for running and managing the application. The case study is the Enterprise Student Record & Academic Management Platform, a Flask application that manages students, courses, enrollments, marks and GPA.

Everything described here was carried out on a single AWS EC2 server controlled from MobaXterm. The companion step-by-step report holds the commands and screenshots; this document explains why each step exists.

> **CORE IDEA: CAMS**
> DevOps rests on four values: **C**ulture, **A**utomation, **M**easurement and **S**haring.
>
> The practical goal is to shorten the time from a code change to a working release, while making releases more frequent, more stable and quicker to recover when something fails.

2. The Business Need for DevOps

2.1 The Problem with Traditional Delivery

In the traditional model, developers wrote features and handed a finished package to a separate operations team. The two groups had opposite goals: developers were rewarded for change, operations for stability. The handover point became known as the "wall of confusion", and most delivery problems happened there.

- **Environment drift:** the program runs on the developer's machine but fails on the server because of a different library version or setting ("it works on my machine").
- **Integration problems:** work merged only at the end of a long project produces large conflicts that take days to untangle.
- **Manual releases:** copying files and editing settings by hand is slow, undocumented and easy to get wrong.
- **Slow recovery:** when a release fails, there is no quick, tested way to go back to the previous working version.

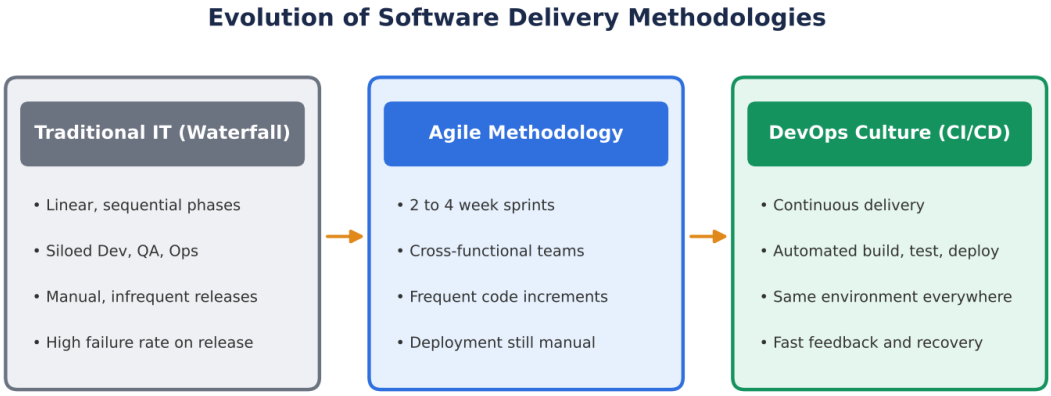


Figure: Evolution from Traditional IT to Agile and DevOps

2.2 Traditional IT vs. Agile vs. DevOps

Agile shortened planning into sprints and improved communication between developers and testers, but it largely stopped at "the code works". DevOps extends the same idea across deployment and operation, so the whole path to the user is automated and repeatable.

Dimension	Traditional (Waterfall)	Agile	DevOps (CI/CD)
Release rhythm	Months or quarters	Every 2 to 4 weeks	Continuous, on demand
Team structure	Separate Dev, QA and Ops	Cross-functional Dev and QA	Dev, QA and Ops share ownership
Deployment	Manual checklists and scripts	Partly automated	Fully automated pipeline
Environments	Frequently different	Staging differs from production	Identical, through containers
Testing	Manual, after development	Per sprint	Automatic on every commit
Failure impact	Large; hard rollback	Moderate	Small; quick rollback

3. The DevOps Lifecycle

The DevOps lifecycle is drawn as a loop because it never ends: what is learned from monitoring a running system feeds straight back into the next round of planning. The eight phases below also map directly to the tools used in this project.

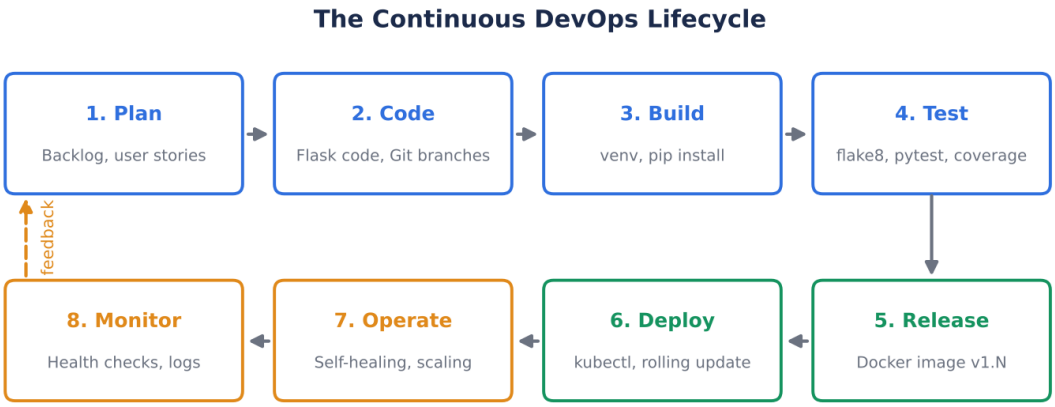


Figure: The continuous DevOps lifecycle and the tools used at each phase

3.1 The Eight Phases

- **Plan:** requirements are turned into small, trackable tasks. In our project the work was split into six topics, one Git branch each.
- **Code:** features are written in Python and Flask on short-lived feature branches, so unfinished work never disturbs the main branch.
- **Build:** dependencies are installed into a clean virtual environment from requirements.txt, proving the project can be rebuilt from scratch.
- **Test:** flake8 checks code style and pytest runs 42 automated tests. A failing test stops the pipeline immediately.
- **Release:** the validated application is packaged into a Docker image tagged with a version, such as v1.0.0 or the Jenkins build number.
- **Deploy:** Kubernetes applies the manifest files and replaces old pods with new ones using a rolling update.
- **Operate:** Kubernetes keeps three replicas running and restarts any that fail.
- **Monitor:** the /api/health endpoint, container health checks and pod status reveal whether the system is well.

> **SHIFT-LEFT TESTING**
> The later a defect is found, the more it costs to fix. A bug caught by an automated test minutes after the commit costs almost nothing; the same bug found by users in production costs time, trust and money.
> Running tests on every change moves quality checks "to the left", early in the lifecycle.

4. Version Control with Git and GitHub

4.1 Principles of Distributed Version Control

A version control system records every change to a project, so any earlier state can be recovered, any change can be traced to its author, and several people can work in parallel without overwriting each other.

Git, created by Linus Torvalds, is a distributed system: every clone holds the complete history, so most operations run locally and do not need a network connection.

4.2 The Four Areas of Git

- **Working directory:** the project files on disk, where editing happens.
- **Staging area (index):** a preparation zone. `git add` places chosen changes here, so a commit contains exactly what was intended.
- **Local repository (.git):** the hidden database of commits. `git commit` stores a permanent snapshot with a unique SHA-1 hash.
- **Remote repository:** the shared copy on GitHub. `git push` uploads commits, `git pull` downloads them.

4.3 Git vs. GitHub

Criterion	Git	GitHub
What it is	Open-source version control tool	Cloud platform that hosts Git repositories
Where it runs	Locally on the computer	On the web, with a browser interface and APIs
Internet needed	No, for commits, branches and logs	Yes, to push, pull and review
Main features	Branching, merging, history, diff	Pull requests, code review, issues, Actions
Role in CI/CD	Records code and pipeline files	Stores the code Jenkins builds from

4.4 Branching Strategy Used in This Project

A branching strategy keeps unstable work away from the code that must always work. We followed a feature-branch workflow: the **main** branch always holds working code, and every topic is built on its own branch, reviewed through a **Pull Request (PR)** and then merged. Branches were kept after merging so the history shows each stage of the project.

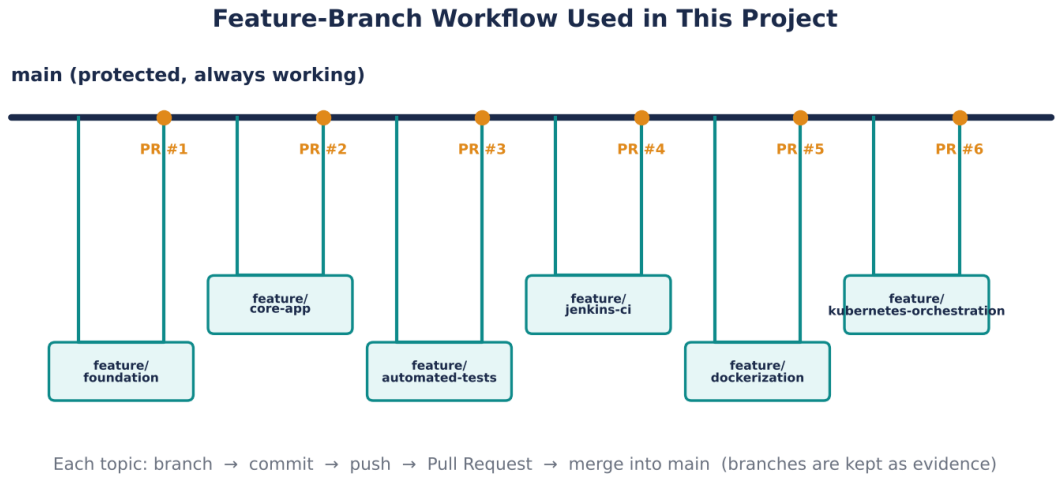


Figure: Feature-branch workflow with six topic branches and six pull requests

Branch	What it delivers
feature/foundation	Flask application skeleton, requirements, health endpoint
feature/core-app	Models, business rules, REST API and web dashboard
feature/automated-tests	42 pytest tests with an 85% coverage gate

Branch	What it delivers
feature/jenkins-ci	Jenkinsfile with four continuous integration stages
feature/dockerization	Dockerfile, .dockerignore and Docker build stage
feature/kubernetes-orchestration	Kubernetes manifests and deployment stage

A pull request is a formal request to merge one branch into another. It gives a place to review the changes, see what was modified and record the decision. Other common strategies include a develop branch for integration, release branches for stabilising versions and hotfix branches for urgent production fixes; for a small project like this one, feature branches into main are sufficient.

5. Continuous Integration with Jenkins

5.1 What Continuous Integration Means

Continuous Integration (CI) is the practice of merging small changes into the shared code often, and verifying every change automatically with a build and a test run. Without CI, people work apart for weeks and then face "integration hell": large conflicts, incompatible assumptions and failures that nobody can trace to a single change. With CI, a problem is found within minutes of the change that caused it, while it is still small and fresh in the author's mind.

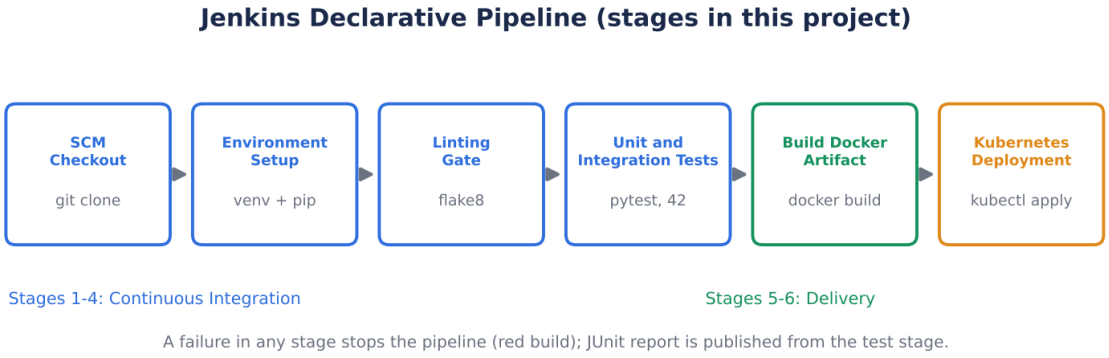


Figure: Jenkins pipeline stages used in this project

5.2 Pipeline Stages in This Project

Stage	What happens	Tool
1. SCM Checkout	Jenkins downloads the latest code from GitHub	Git
2. Environment Setup & Dependencies	A clean virtual environment is created and requirements.txt is installed	python venv, pip
3. Code Linting & Syntax Gate	Code style and syntax errors are checked without running the program	flake8
4. Automated Unit & Integration Tests	42 tests run; a JUnit XML report is produced; coverage must reach 85%	pytest, pytest-cov
5. Build Docker Artifact	The application is packaged into a tagged Docker image	Docker
6. Kubernetes Deployment	Manifests are applied and the rollout is verified	kubectl, Minikube

The first four stages form the CI part and were added through the jenkins-ci branch; the last two were added with the Docker and Kubernetes branches, so the same pipeline file grew with the project. If any

stage fails, the pipeline stops and the build is marked red, so a broken change never reaches the packaging or deployment stages.

5.3 Jenkins Architecture

Jenkins follows a controller and agent model. The **controller** serves the web interface, stores job configuration and schedules builds. **Agents** are worker machines that run the actual build commands, which allows many builds to run in parallel on separate machines. In this project a single server plays both roles, which is sufficient for a small team and keeps the setup simple. The Jenkins service runs as the same operating-system user that owns Docker and Minikube, so a pipeline can call those tools directly.

5.4 The Jenkinsfile: Pipeline as Code

A Jenkinsfile is a text file, stored in the repository, that describes the pipeline. Keeping it under version control means the build process is reviewed, versioned and restored like any other code. The project uses the **declarative** syntax, whose main building blocks are:

- **pipeline / agent:** the outer structure and where the pipeline runs.
- **environment:** variables shared by all stages, such as the image name and the version tag (v1. plus the build number).
- **stages / stage / steps:** the ordered work, shown as the coloured boxes in the Jenkins Stage View.
- **post:** actions after the run, such as publishing the test report and cleaning the workspace.

5.5 Triggering Builds and Test Reporting

Jenkins can start a build in two main ways. A **webhook** lets GitHub notify Jenkins instantly after a push. **Poll SCM** makes Jenkins check the repository on a schedule and build when it finds a new commit. Our EC2 server is reached through a changing public address, so polling every two minutes (H/2 * * * *) was used; it needs no inbound connection from GitHub and gives the same result with a short delay.

The test stage writes results in the standard JUnit XML format, and Jenkins' junit step turns that file into a Test Result page showing how many tests passed or failed. A quality gate on top of this requires at least 85% code coverage, so tests that do not exercise the code cannot pass the build.

6. Containerization with Docker

6.1 Virtual Machines vs. Containers

A virtual machine (VM) emulates a whole computer, including its own guest operating system, so each VM needs gigabytes of memory and minutes to start. A container isolates only the application and its libraries and shares the host's operating-system kernel, so it is measured in megabytes and starts in seconds. Two Linux features make this possible: **namespaces**, which give each container its own private view of processes, network and file system, and **control groups (cgroups)**, which limit the CPU and memory a container can use.

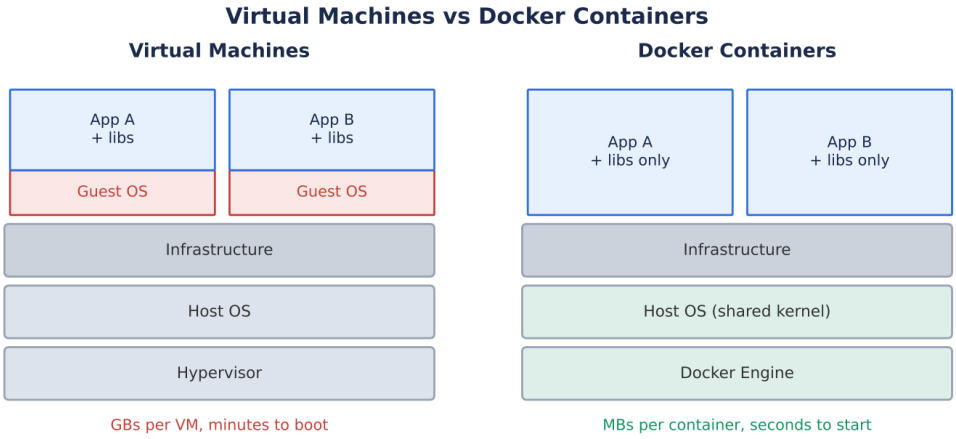


Figure: Architecture of virtual machines compared with Docker containers

Attribute	Virtual Machine	Docker Container
Virtualization level	Hardware (hypervisor)	Operating system (Docker Engine)
Kernel	Each VM has its own guest OS	Shares the host kernel
Start-up time	Minutes	Seconds or less
Size	Gigabytes	Tens to hundreds of megabytes
Density	Few per server	Many per server
Portability	Tied to hypervisor formats	Runs the same wherever Docker runs

6.2 Images, Layers and Containers

A **Docker image** is a read-only template built from the instructions in a **Dockerfile**. Each instruction creates a layer, and layers are cached and shared. If a step has not changed, Docker reuses it, so rebuilds are fast; this is why the Dockerfile copies requirements.txt and installs dependencies before copying the application code. A **container** is a running instance of an image, with a thin writable layer on top. The image never changes, which is what guarantees that the same package runs the same way in every environment.

6.3 The Dockerfile of This Project

Instruction	Purpose
FROM python:3.11-slim	Small official Python base image
Install curl	Needed by the container health check
Create and switch to appuser	Run the application as a non-root user for safety
COPY requirements.txt, then pip install	Install dependencies in a cached layer
COPY application code	Add the app, templates and database module
EXPOSE 5000	Document the port the application listens on
HEALTHCHECK	Periodically call /api/health; Docker reports healthy or unhealthy
CMD python run.py	Command that starts the application

A **.dockerignore** file keeps the virtual environment, Git data, test reports and cache files out of the image, so it stays small and clean. Images are labelled with **tags**: v1.0.0 is a fixed release, while the Jenkins pipeline tags each build as v1.N (N being the build number) and also latest, so every version can be traced to the build that produced it.

7. Container Orchestration with Kubernetes and Minikube

7.1 Why Orchestration Is Needed

Docker runs containers, but a real service needs more than starting them. Someone must restart containers that crash, run several copies to share load, replace versions without interrupting users, and give clients one stable address even though containers come and go. Kubernetes automates all of this. You describe the state you want, for example "three copies of this image, always", and Kubernetes keeps reality matching that description.

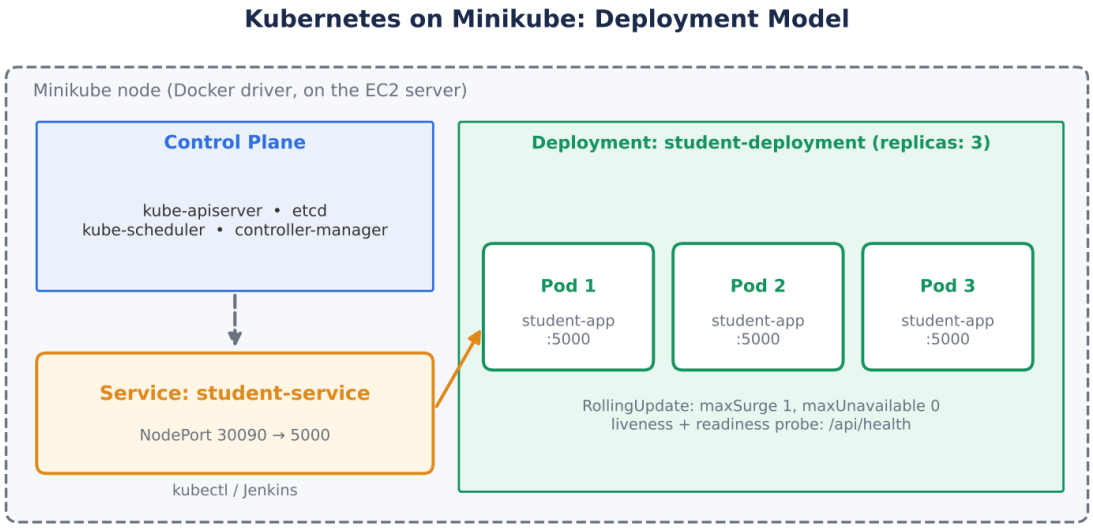


Figure: Kubernetes deployment model on Minikube

7.2 Architecture and Core Objects

A Kubernetes cluster has a **control plane**, which makes decisions, and **worker nodes**, which run the containers.

- **kube-apiserver:** the front door; every kubectl command goes through it.
- **etcd:** the key-value store that holds the cluster's state.
- **kube-scheduler:** chooses which node should run each new pod.
- **kube-controller-manager:** compares desired and actual state and corrects differences, such as a missing pod.
- **kubelet and kube-proxy:** the node agent that runs pods, and the network component that routes traffic to them.

The objects used in this project are:

- **Pod:** the smallest unit, wrapping one running container.
- **Deployment:** keeps the requested number of identical pods running and manages updates through ReplicaSets.
- **Service:** a stable address and load balancer in front of the changing pods.

7.3 The Manifests of This Project

Setting	Value	Why it matters
Deployment name	student-deployment	Name used with kubectl commands

Setting	Value	Why it matters
Replicas	3	Three identical pods share traffic and survive the loss of one
Update strategy	RollingUpdate, maxSurge 1, maxUnavailable 0	A new pod must be ready before an old one is removed, so there is no downtime
Liveness probe	GET /api/health	Restarts a container that stops responding
Readiness probe	GET /api/health	Sends traffic only to pods that are ready
Resources	request 128Mi / 100m, limit 256Mi / 250m	Reserves capacity and prevents one pod from using everything
Service type	NodePort 30090 to port 5000	Makes the application reachable from outside the cluster

7.4 Self-Healing and Rolling Updates

Self-healing was demonstrated by deleting one pod by hand. The Deployment immediately noticed that only two of three pods existed and created a replacement, so the service stayed available throughout. **Rolling updates** work in the same spirit: when Jenkins applies a manifest with a new image tag, Kubernetes starts one new pod, waits until its readiness probe passes, stops one old pod, and repeats until all three run the new version. Kubernetes records each revision, so a faulty release can be reversed with a rollback command.

7.5 Minikube-Specific Details

Minikube runs a complete single-node Kubernetes cluster inside a Docker container on one machine, which suits learning and testing. Because it has its own image store, an image built with the normal Docker daemon is invisible to it. The project solved this in two ways: for manual builds, the `minikube docker-env` command points Docker at Minikube's daemon; inside the Jenkins pipeline, `minikube image load` copies the freshly built image into the cluster. The deployment sets imagePullPolicy to IfNotPresent so Kubernetes uses that local image instead of trying to download it. Since a NodePort lives on the Minikube node rather than on the EC2 server itself, a `kubectl port-forward` command was used to open the application in a browser.

8. The Integrated DevOps Ecosystem

Git, Jenkins, Docker and Kubernetes are most valuable when they work as one chain, where each tool hands its result to the next without manual steps.

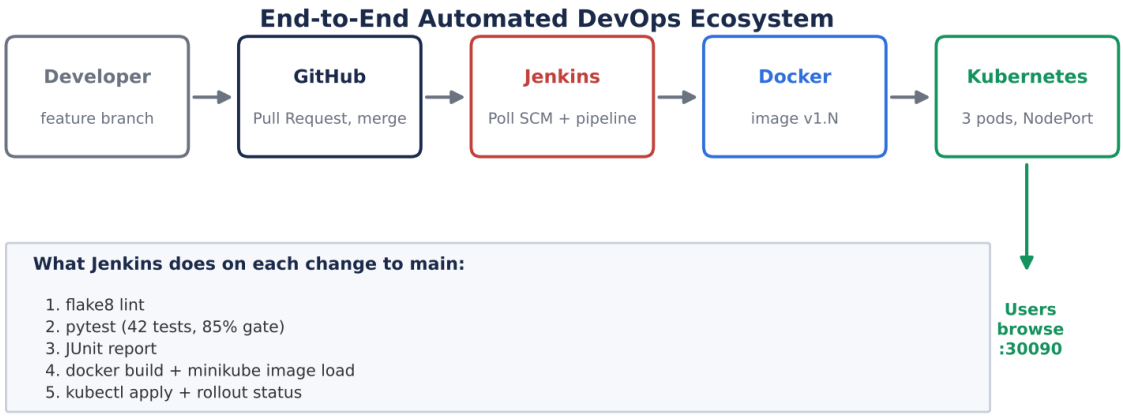


Figure: End-to-end automated DevOps ecosystem of this project

8.1 From Commit to Running Application

- **Step 1:** the developer pushes a feature branch to GitHub and opens a pull request.
- **Step 2:** after review, the pull request is merged into main.
- **Step 3:** Jenkins detects the new commit on main and checks out the code.
- **Step 4:** Jenkins installs dependencies, runs flake8 and executes the 42 pytest tests with the coverage gate.
- **Step 5:** Jenkins builds a Docker image tagged v1.N.
- **Step 6:** Jenkins loads the image into Minikube and applies the Kubernetes manifests.
- **Step 7:** Kubernetes performs a rolling update, checks readiness, and routes users through the NodePort service.
- **Step 8:** health checks and pod status show whether the new version is healthy, closing the feedback loop.

In an enterprise setting, step 5 would also push the image to a container registry such as Docker Hub or Amazon ECR so that any cluster can pull it. This project uses a single machine, so the image is loaded into Minikube directly and no registry is needed.

8.2 Application Design: What the Pipeline Delivers

The application at the centre of the pipeline is a Flask web service with a REST API and a dashboard. Its business rules are enforced in a service layer and verified by the automated tests, which is what makes safe, frequent releases possible.

Area	Rule or feature
Credit limit	A student may enroll in at most 18 credits per semester
Timetable	Two courses in the same slot cannot be taken together
Capacity	A course cannot exceed its capacity; a student cannot enroll twice in the same course
Marks and grades	Marks are 0 to 100 and map to grade points: A=4, B=3, C=2, D=1, F=0
GPA	Calculated as a credit-weighted average
Health endpoint	/api/health returns status UP; used by Docker and Kubernetes probes

8.3 Trainer Case Study vs. Student Project

Aspect	Trainer (Library Platform)	Student (Student Record Platform)
Domain	Books, borrowers, loans, fines	Students, courses, enrollments, marks
Key rules	Duplicate ISBN, membership, checkouts	Credit limit, slot conflict, capacity, GPA
Test files	test_book_service, test_borrow_return	test_student_service, test_enrollment, test_marks_mgmt, test_api_endpoints
Docker image	library-platform:v1.0.0	student-academic-platform:v1.0.0
Deployment	Minikube	Minikube, 3 replicas, NodePort 30090

9. Challenges Encountered and Lessons Learned

- **Permissions between tools:** the pipeline can only call Docker and Minikube if the Jenkins service runs as the user that has access to them, so the service was configured that way during setup. Lesson: automation tools need exactly the access the pipeline uses, and no more.
- **Images and Minikube:** Minikube keeps its own image store, so an image built with the host's Docker is invisible to the cluster and pods would fail with an image pull error. The fix was to build inside Minikube's Docker (docker-env) and to use `minikube image load` in Jenkins. Lesson: know where each tool stores its images.
- **Project in a repository subfolder:** the project lives inside an existing repository, so Jenkins needed the Script Path student-academic-platform/Jenkinsfile and the stages run inside that folder. Lesson: pipeline paths must match the repository layout.
- **Port conflict:** Jenkins uses port 8080, which is also the default address kubectl tries when no cluster is configured, so kubectl showed the Jenkins login page. Lesson: read error output carefully; the cause may be a different service.
- **Cost and security:** the cloud server accrues charges while running and access tokens are secrets. Lesson: restrict ports to your own address, never share tokens or key files, and terminate resources after use.

10. Conclusion

The project shows that a small application can be delivered with the same discipline used by large engineering teams. Git and pull requests protected the main branch and recorded every step. Jenkins turned building and testing into an automatic, repeatable gate. Docker made the application portable and identical in every environment. Kubernetes kept it available, replaced versions without downtime and recovered from failure on its own. Together they replace manual, error-prone releases with a pipeline that is traceable, repeatable and fast, which is the central goal of DevOps.

Glossary of Key Terms

Term	Meaning
CI/CD	Continuous Integration and Continuous Delivery: automatic build, test and release of every change
Pull Request	A request to merge one branch into another, with review
Jenkinsfile	Text file that defines a Jenkins pipeline as code
JUnit report	Standard XML test result file that Jenkins turns into a report page
Docker image	Read-only template used to create containers

Term	Meaning
Container	Running, isolated instance of an image
Pod	Smallest deployable unit in Kubernetes
Deployment	Kubernetes object that keeps a set number of pods running and manages updates
Service / NodePort	Stable network address for pods; NodePort exposes it on a fixed port of the node
Rolling update	Replacing pods gradually so the service never goes down
Self-healing	Automatic replacement of failed pods